Closing the Gap: A Unified Machine Learning Approach to Pull Request Prioritization and Reviewer Recommendation

Niaz Rahman, Antara Arifa Mullick, and Labibah Hoque

***Abstract*— Pull request review delays are a common problem in modern software development. They usually happen for two reasons: pull requests are not prioritized well, and reviewers are not assigned effectively. Most existing tools solve only one of these problems at a time, which creates inefficiencies because the two decisions are closely connected. We propose a unified machine learning system that handles both together, ranking pull requests by urgency and recommending suitable reviewers at the same time. The system uses a set of features covering reviewer expertise, collaboration history, workload, and text similarity, and is evaluated on a real-world open-source repository.**

***Keywords*— Pull request prioritization, reviewer recommendation, machine learning, software repository mining, code review automation, latency prediction.**

I. Introduction

Modern software development is deeply collaborative. Platforms such as GitHub, GitLab, and Bitbucket rely on the pull based development model, where contributors submit pull requests (PRs) for peer review before their changes are merged into the main codebase [1]. This workflow improves code quality, encourages discussion, and produces a traceable history of project evolution.

Despite these advantages, pull request review latency, the time between a PR being opened and being closed or merged, is a well documented bottleneck in many projects [2]. As repositories grow in activity and contributor base, review queues become increasingly difficult to manage. Important bug fixes can sit unreviewed for days, first time contributors may lose motivation waiting for feedback, and security patches may be delayed at critical moments.

A. The Dual Nature of the Latency Problem

PR latency is driven by two connected but usually separately studied problems. The first is *poor prioritization*: maintainers frequently process pull requests in submission order or based on familiarity with the contributor, causing urgent changes to be overlooked. The second is *ineffective reviewer assignment*: reviewers are often selected manually or through simple file overlap heuristics, leading to assignments that are mismatched in terms of expertise, availability, or current workload.

Niaz Rahman (210042159), Antara Arifa Mullick (210042132), and Labibah Hoque (210042121), Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. Supervised by Lutfun Nahar Lata (Assistant Professor) and Azam Hossain (Associate Professor), Department of Computer Science and Engineering, Islamic University of Technology.

These two problems amplify each other. When the wrong reviewer is assigned to a high priority PR, the expected speed benefit disappears. Conversely, when the review queue is long and poorly ordered, even the best matched reviewer cannot address the most urgent issues first. Together, they create a feedback loop that inflates overall latency.

B. Why Existing Approaches Fall Short

Most existing research treats reviewer assignment and PR prioritization as independent tasks. Systems such as RevFinder [3] and CHRev [4] focus on identifying suitable reviewers based on file ownership or review history, while tools like ARPrioritizer [10] rank pull requests by urgency without considering who will actually perform the review. This separation is problematic in practice: optimizing reviewer selection independently of the queue state, or ranking PRs without accounting for reviewer availability, produces recommendations that do not reflect the true complexity of managing a live review pipeline.

C. Our Approach and Contributions

This paper presents a unified machine learning pipeline that addresses pull request prioritization and reviewer recommendation jointly. Given a repository's historical PR data, the system produces a single output: a priority ordered list of pull requests, each with an assigned reviewer. The key design insight is that the two decisions are not independent, the urgency of a PR should influence which reviewer is selected, and the current workload of reviewers should influence how PRs are prioritized. By solving them together in a *queue aware* fashion, that is, by conditioning every assignment and ordering decision on the live, continuously updated state of the review queue and of reviewer workloads, rather than on a single static snapshot computed once in advance, the system avoids the inconsistencies that arise when the tasks are handled separately.

The specific contributions of this work are:

- A three phase unified pipeline covering feature engineering, predictive modeling, and joint optimization for PR prioritization and reviewer assignment.
- A *dual signal priority score* that fuses a PR's predicted acceptance probability with its predicted likelihood of receiving a timely integrator response into a single urgency value, which is then used to order the review queue. We adopt the two signal formulation of Khan et al. [10];

our contribution is to couple this score with reviewer assignment rather than to treat it in isolation.
- A *queue aware* greedy reviewer assignment strategy. As defined above, *queue aware* means each assignment is conditioned on the live state of the queue and on reviewers' current *workload*, where a reviewer's workload is quantified as the number of active (open, not yet completed) review assignments currently held by that reviewer. Assigning a PR increments the chosen reviewer's workload immediately, so the update propagates to and influences every subsequent assignment in the queue.

D. Scope and Organization

The system targets repositories that use GitHub style pull request workflows and have sufficient historical data for supervised learning. The evaluation is conducted on the Apache Flink repository (2014 to 2020), a large and active open source project with diverse subsystems.

II. Related Work

This paper addresses pull request review latency from the perspective of two interrelated problems: identifying the most suitable reviewer for a pull request, and determining which pull request in the queue deserves attention first. The related work in this section therefore covers automated reviewer recommendation and pull request prioritization, the two research threads most directly relevant to our system.

A. Reviewer Recommendation

Selecting appropriate reviewers for pull requests is a time consuming task that can delay code integration by up to twelve days in active repositories [5]. The stakes are not merely speed: in an empirical study at Microsoft, Bosu et al. [8] found that the usefulness of a code review depends heavily on the reviewer's prior familiarity with the changed code, which provides empirical grounding for expertise based recommendation. Early automated systems addressed this by exploiting file ownership heuristics: a developer who had previously modified a file was assumed to be a suitable reviewer for changes to that file. While this intuition is broadly valid, it has well documented limitations, past contributions are treated as equally informative regardless of age, and the social relationship between the PR author and candidate reviewers is ignored entirely.

More recent work has extended file based approaches along two complementary directions. Moosareza and Heydarnoori [5] propose combining review expertise, commit expertise, and a collaboration score into a single recommendation model, where each signal is weighted by a cube root time decay function that reduces a candidate's weight as the number of days since their last relevant activity grows. The cube root form is chosen because it captures the non linear nature of knowledge decay more faithfully than a linear decay while remaining computationally tractable. Evaluated on five GitHub projects including Apache Flink, the method achieves 46% top 1 precision, a meaningful improvement over file only baselines.

Yu et al. [6] approach the same problem from a social network perspective. They observe that a reviewer who has repeatedly interacted with a contributor, through review comments, co authorship, or shared discussion, tends to respond faster and provide higher quality feedback than one selected on code familiarity alone. To exploit this, they construct a *Comment Network* (CN): a directed weighted graph in which the weight of the edge from a contributor to a reviewer grows with the number of past interactions between them and decays exponentially with the time elapsed since each interaction, so that frequent and recent collaboration is rewarded most strongly. For known contributors, suitable reviewers are identified by breadth first graph traversal; for newcomers with no history, Louvain community detection places them in the most relevant reviewer cluster. Evaluated on approximately 17,715 pull requests across ten GitHub projects, the Comment Network achieves 78% top 1 precision and 77% recall at top 10, substantially above file overlap baselines.

A third line of work blends these signals. Xia et al. [7] propose TIRR, which augments a topic model of the changed files with an explicit measure of reviewer influence, allowing the recommender to break ties and handle cold start cases that pure file overlap methods treat poorly. This reinforces a recurring theme in the literature: file ownership, expertise decay, social interaction, and topic relevance are each partial signals that are most effective when combined.

Despite these advances, all of these systems share a common limitation: they focus exclusively on *who* should review a pull request without considering *which* pull request should be reviewed first. The result is that recommendations are made in isolation from the state of the review queue, which can render even the best reviewer assignment ineffective if the PR itself is deprioritized.

B. Pull Request Prioritization

Managing high volumes of pull requests is a well documented bottleneck in open source software development. When queues grow large and unranked, urgent contributions, security patches, critical bug fixes, and submissions from new contributors at risk of disengaging, are frequently delayed behind less important items. Research into automated PR prioritization has sought to address this by predicting which pull requests most require immediate attention.

The predictive signals used for urgency estimation rest on a body of empirical work on contribution acceptance. Weissgerber et al. [12] showed that smaller patches are accepted more often and more quickly than large ones, establishing patch size as an early predictor of contribution fate, while Soares et al. [9] characterized the factors behind pull request rejection even among core team developers. Together these findings motivate treating acceptance probability as a learnable signal rather than a fixed property of a contributor.

Khan et al. [10] observe that no single predictive signal is sufficient for reliable urgency estimation: a pull request may be highly likely to be accepted yet receive no integrator attention for days, while a PR that elicits a fast response may ultimately be rejected. To capture both dimensions, they introduce ARPrioritizer, which trains two independent XGBoost

classifiers, one predicting the probability that the pull request will eventually be accepted and one predicting the probability that an integrator will respond within 24 hours, and combines the two predicted probabilities into a single urgency score for ranking the queue. Evaluated on 278,418 pull requests from 41 GitHub projects, the system achieves AUC scores of 0.919 and 0.893 for the two classifiers respectively, significantly outperforming single signal baselines.

Raskin et al. [11] approach prioritization from a usability perspective. They note that even a well ordered queue is unhelpful if it is not surfaced within the developer's natural workflow. Their tool, PRECOG, is a Visual Studio Code plugin that ranks open pull requests by predicted review difficulty using static analysis metrics from SonarCloud and CodeQL, presenting rankings as radar charts directly inside the IDE. In a user study on the Java Spring Framework repository, developers found PRECOG effective for reducing decision latency when triaging the review queue.

A complementary, intervention based view is taken by Mirsaeedi and Rigby [13], whose tool Nudge identifies overdue pull requests and automatically nudges the responsible parties toward completion, accelerating PRs that have stalled in the queue. Nudge escalates stalled work rather than ordering the queue from scratch, but it shares the limitation of the prioritization systems above: it surfaces or escalates PRs without deciding *who* should review them.

These systems, however, leave reviewer assignment entirely to human judgment. They produce ranked PR lists without considering who is actually available to handle the most urgent items, and none accounts for the propagating effect of assignment decisions: assigning a reviewer to the highest priority PR changes their availability for every subsequent PR in the queue, a dependency that isolated prioritization systems cannot represent.

III. Methodology

The overall architecture of our unified pipeline is shown in Fig. 1. It is organized into three sequential phases, data preparation and analysis, modeling and scoring, and optimization and assignment, that together transform a repository's raw pull request history into a priority ordered list of pull requests, each paired with a recommended reviewer.

A. Dataset and Evaluation Protocol

The system is evaluated on the Apache Flink GitHub repository, covering pull requests, reviews, commits, comments, and participant activity from 2014 to 2020. The raw data contains 10,819 pull requests and a candidate pool of 925 distinct reviewers who have contributed at least one review to the project.

A critical methodological requirement is the avoidance of temporal data leakage: the model must never be trained on information that would not have been available at the time a given pull request was actually opened. To enforce this, pull requests are sorted by creation date and split at the 80th percentile, so that the oldest 80% of pull requests (8,655 PRs) form the training set and the most recent 20% (2,164 PRs) form the held out evaluation period. Within the evaluation period, only pull requests that have a recorded ground truth reviewer in the review history and a valid, non negative first review latency are retained, since both are required to score the model against reality. This filtering yields 1,557 unique evaluation pull requests, corresponding to 2,439 evaluation rows once pull requests with more than one historical reviewer are accounted for.

B. Phase One: Data Preparation and Feature Engineering

The first phase constructs the feature representations used by both downstream models. Reviewer level history is aggregated to compute each candidate's total review count, mean review latency, and acceptance rate. File level history is used to build a file to reviewer mapping, from which a file overlap score between a candidate reviewer and a new pull request is computed as the proportion of changed files the candidate has previously reviewed. An analogous directory overlap score is computed at the directory level to provide a coarser grained expertise signal that remains informative even when exact file overlap is low.

Commit history is processed separately to derive a commit expertise score, capturing whether a candidate has authored commits touching the same files as the pull request under evaluation, following the intuition established in Paper 1 that code authorship is a strong proxy for review competence. Author reviewer affinity pairs are extracted from past pull requests to quantify how often a given author and reviewer have worked together previously. A reviewer specialization score is also computed, measuring the proportion of a pull request's files that fall within a reviewer's single most frequently reviewed directory, weighted by how concentrated that reviewer's expertise is in that directory.

Following Yu et al. [6], a comment network is constructed from the participant and review comment data, producing a directed weighted graph between contributors and reviewers. Edge weights decay with time, so that recent interactions are weighted more heavily than older ones, and the resulting collaboration network score is included as a feature representing social familiarity independent of code level overlap.

Recency of a candidate's most recent review activity is transformed using a cube root decay function, following the non linear decay formulation of Moosareza and Heydarnoori [5], so that very recent activity is rewarded strongly while the marginal difference between two stale candidates is compressed. Two cross interaction features are also constructed: the product of file overlap and recency, and the product of commit expertise and author affinity, to allow the downstream model to exploit conditions under which two moderately informative signals jointly indicate a strong match. A reviewer workload feature, the number of pull requests a reviewer has open in the preceding 30 day window, is computed across the entire timeline rather than only at evaluation time, so that it reflects genuine historical workload rather than a value available only in retrospect. In total, sixteen numeric features are engineered per candidate pull request pair, together with a text similarity representation built from term frequency inverse

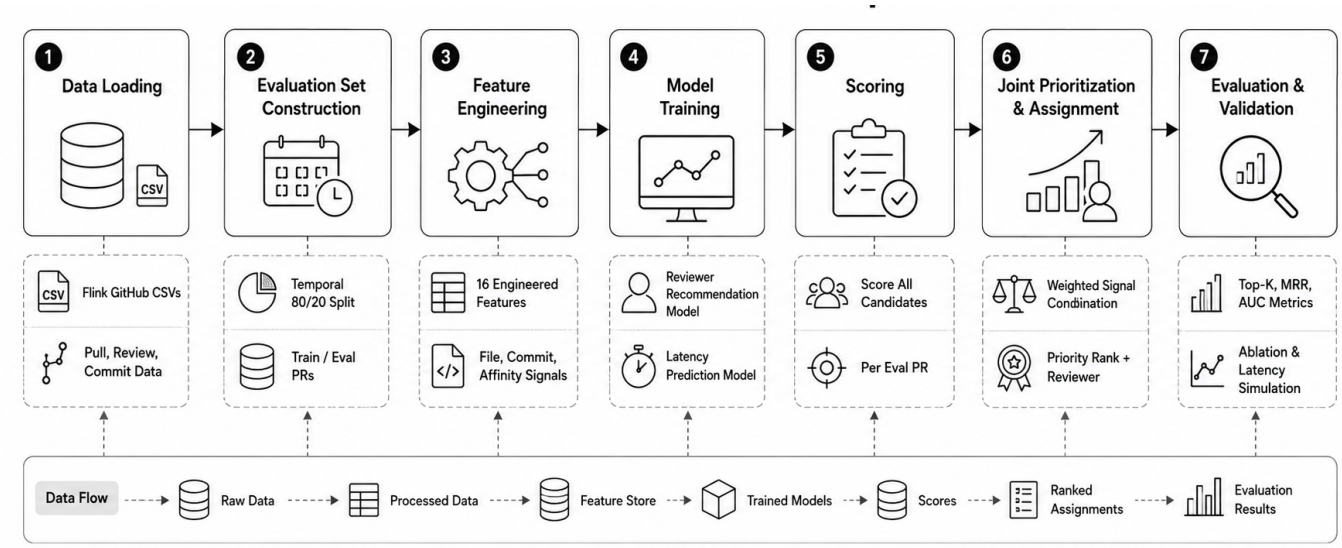


Fig. 1. Overview of the proposed unified pipeline for joint pull request prioritization and reviewer recommendation, comprising data preparation and analysis, modeling and scoring, and optimization and assignment.

document frequency (TF-IDF) vectorization of the pull request title, description, and changed file paths.

C. Phase Two: Modeling and Scoring

The second phase trains two supervised models on the training partition described in Section III-A. The first is a reviewer recommendation model, a Gradient Boosting classifier trained to predict the probability that a given candidate is the correct reviewer for a given pull request, functionally equivalent to the XGBoost formulation used in prior work [10]. Positive training pairs are drawn from pull requests with a confirmed historical reviewer, and negative pairs are sampled at a five to one ratio to reflect the natural class imbalance of the reviewer assignment problem, where for any given pull request the vast majority of the 925 candidate reviewers are not suitable matches.

The second is a latency prediction model, a Random Forest regressor trained to predict the time, in hours, between a pull request being opened and receiving its first review. Because review latency is heavily right skewed, with the majority of pull requests reviewed within hours but a long tail extending to weeks, the model is trained on a log transformed target, $\log(1+\text{latency})$, and predictions are inverse transformed back to hours at inference time. Beyond pull request size and participant count, the latency model incorporates the recommended reviewer's historical mean latency, the author's historical mean latency, and temporal features capturing the day of week, hour of day, and whether the pull request was opened on a weekend, since review responsiveness in practice fluctuates with these calendar effects.

At evaluation time, every pull request in the held out evaluation period is scored against all 925 candidate reviewers using both trained models, producing, for each pull request, a ranked list of candidates by predicted suitability and a single predicted latency value.

D. Phase Three: Optimization and Assignment

The third phase combines the outputs of phase two into a single joint priority score and performs queue aware reviewer assignment. For each pull request, a priority score is computed as a weighted combination of seven signals: the learned suitability probability from the recommendation model, an urgency term derived from the inverse of predicted latency, file level expertise, commit level expertise, directory level expertise, the collaboration network score, and reviewer specialization, combined with a negative weight applied to the current workload of the candidate being considered. Formally, for a pull request PR and a candidate reviewer r, the joint priority score is defined as

$$\begin{aligned}\text{Priority}(PR, r) = \alpha \cdot S(r) + \beta \cdot U(PR) + \gamma \cdot F(r) \\ + \delta \cdot C(r) + \eta \cdot D(r) + \zeta \cdot N(r) \\ + \iota \cdot Z(r) - \mu \cdot W(r)\end{aligned} \quad (1)$$

where $S(r)$ is the learned suitability probability assigned to candidate r by the reviewer recommendation model, $U(PR)$ is the urgency term derived from the inverse of the predicted latency, $F(r)$ is the file level expertise score, $C(r)$ is the commit level expertise score, $D(r)$ is the directory level expertise score, $N(r)$ is the collaboration network score, $Z(r)$ is the reviewer specialization score, and $W(r)$ is the current workload of candidate r at the time the score is computed. The coefficients α, β, γ, δ, η, ζ, and ι are non negative weights applied to their respective signals, and μ is a non negative weight applied to the workload penalty, so that candidates with a heavier current workload receive a correspondingly lower priority score. In this work the weights are set to $\alpha = 0.25$, $\beta = 0.15$, $\gamma = 0.15$, $\delta = 0.12$, $\eta = 0.08$, $\zeta = 0.10$, $\iota = 0.10$, and $\mu = 0.05$, reflecting the relative emphasis placed on the learned suitability signal while still allowing expertise, collaboration, and workload signals to materially

influence the final ranking; the contribution of each weighted term to overall performance is examined in the ablation study reported in Section IV.

Because $W(r)$ depends on the current, continuously updated workload of candidate r rather than on a static snapshot, Priority(PR, r) in Equation 1 is recomputed, for the relevant candidates, every time a new assignment is made. Pull requests are then processed in descending order of priority score, and for each pull request the highest scoring available reviewer is assigned. Critically, each assignment immediately increments that reviewer's workload counter, so that subsequent, lower priority pull requests in the same queue are scored against an updated availability state rather than a static snapshot computed once at the start. This is the mechanism by which the system satisfies the queue aware property described in Section II: the urgency of a pull request and the workload of a reviewer are never evaluated independently of one another.

E. Evaluation Methodology

The system is evaluated along three complementary lines. First, standard reviewer recommendation metrics, area under the receiver operating characteristic curve (AUC ROC), top k accuracy for k in 1, 3, 5, and 10, and mean reciprocal rank (MRR), are computed against a normalized candidate pool of 54 reviewers, matching the pool size used in prior file overlap work [5] for fair comparison, as well as against the full 925 candidate pool to characterize performance under realistic, unrestricted conditions. Second, an ablation study systematically removes individual components of the joint priority score, the learned model, the collaboration network signal, commit expertise, and reviewer specialization, to isolate the contribution of each. Third, prioritization quality is evaluated independently of reviewer identity, using Spearman rank correlation between assigned priority rank and actual observed latency, a quintile based latency comparison between the highest and lowest priority pull requests, and normalized discounted cumulative gain (NDCG) of the priority ordering relative to first in first out (FIFO) and random orderings.

IV. Results

A. Reviewer Recommendation Performance

Table I reports reviewer recommendation performance on the held out evaluation set of 1,557 pull requests, comparing a heuristic baseline that always recommends the most historically active reviewer against the proposed model, evaluated on both the normalized 54 candidate pool and the full 925 candidate pool.

On the normalized pool, the proposed model improves top 1 accuracy from 9.4% to 49.8%, more than a five fold improvement, and improves MRR from 0.265 to 0.866. On the full 925 candidate pool, absolute top k accuracy is necessarily lower, since the model must distinguish the correct reviewer from a much larger candidate set, but AUC ROC remains close to its ceiling at 0.999, indicating that the model continues to rank the correct reviewer far above the majority of incorrect candidates even when the pool size grows by more than an order of magnitude.

TABLE I
REVIEWER RECOMMENDATION PERFORMANCE ON THE EVALUATION SET

Metric	Baseline	Ours (54 pool)	Ours (925 pool)
AUC ROC	0.980	0.990	0.999
Top 1 accuracy	9.4%	49.8%	19.3%
Top 3 accuracy	15.6%	89.1%	34.2%
Top 5 accuracy	21.6%	97.1%	39.6%
Top 10 accuracy	38.9%	99.7%	44.3%
MRR	0.265	0.866	0.355

B. Latency Prediction Performance

The latency prediction model achieves a mean absolute error of 61.2 hours on the evaluation set, compared to 133.9 hours for a baseline that predicts each reviewer's historical mean latency directly, a reduction of approximately 54%. Internal validation mean absolute error during training was higher, at 154.5 hours, which is expected given the extreme right skew of the latency distribution and the model's log transformed training objective; the lower evaluation error suggests that the temporal patterns captured by the model generalize reasonably well to the most recent period of project activity.

C. Ablation Study

Table II presents the ablation study, in which individual components of the joint priority score are removed one at a time and the resulting recommendation performance, evaluated on the normalized 54 candidate pool, is reported.

TABLE II
ABLATION STUDY ON THE NORMALIZED 54 CANDIDATE POOL

Configuration	Top 1	MRR	Est. Latency (h)
Full integrated system	48.9%	0.857	57.0
Learned model only	49.8%	0.866	57.3
Expert signals only	37.9%	0.738	54.9
Random reviewer	0.3%	0.010	348.2
Without collaboration network	48.5%	0.853	56.7
Without commit expertise	50.5%	0.877	56.4
Without specialization	48.8%	0.856	57.6

Two findings stand out. First, replacing the learned recommendation model with a random reviewer choice collapses top 1 accuracy from 48.9% to 0.3%, confirming that the learned suitability signal, rather than the hand engineered joint score weighting alone, is responsible for the large majority of recommendation quality. Second, removing commit expertise from the joint score slightly improves top 1 accuracy and MRR rather than harming them. We interpret this as evidence that the Gradient Boosting model already absorbs the predictive content of commit expertise through its own feature set during training, so that re weighting it again in the final joint score introduces mild redundancy rather than additional signal. This is a useful finding for future weight tuning, since it suggests the hand set weight assigned to commit expertise in the joint score could be reduced without loss of accuracy, freeing weight to be reallocated to signals such as the collaboration network, whose removal does measurably reduce performance.

D. Prioritization Quality

Independently of reviewer identity, the quality of the priority ordering itself is evaluated against actual observed latency. The Spearman rank correlation between a pull request's assigned priority rank and its actual review latency is 0.431 ($p < 0.001$), indicating a statistically significant and moderately strong relationship in the expected direction: higher priority pull requests tend to be reviewed sooner.

A quintile based analysis reinforces this result. The top 20% of pull requests by assigned priority have a mean actual latency of 12.9 hours, compared to 237.3 hours for the bottom 20%, an 18.4 fold difference, and an 85.2% reduction relative to the unranked overall mean of 86.9 hours. Normalized discounted cumulative gain of the priority ordering is 0.837, compared to 0.748 for a first in first out ordering and 0.771 for a random ordering, confirming that the learned priority score orders the queue more effectively than either naive baseline, with the largest relative advantage concentrated in the earliest positions of the queue, where NDCG at the top 10 and top 20 positions is more than three times higher than FIFO.

E. Comparison with Related Work

The top 1 precision achieved by the proposed system on the normalized candidate pool, 49.8%, is broadly comparable to or exceeds the 46% top 1 precision reported by Moosareza and Heydarnoori [5] on a file, commit, and collaboration based recommender evaluated across five GitHub projects including Apache Flink, and the AUC ROC of 0.990 is favorable relative to the AUC scores of 0.919 and 0.893 reported by Khan et al. [10] for their dual classifier prioritization system, though direct comparison is approximate given differences in dataset composition, evaluation period, and candidate pool construction across studies.

V. Discussion and Limitations

The results in Section IV support the central premise of this work, that pull request prioritization and reviewer recommendation benefit from being addressed jointly rather than independently. The ablation study shows that no single component of the joint score is solely responsible for performance, and the prioritization quality results show that the learned priority ordering meaningfully separates pull requests that are reviewed quickly from those that are not.

At the same time, several limitations should be acknowledged. First, the strength of the engineered features, particularly file overlap and recency, is itself partly a product of how the ground truth label is constructed from historical review assignments, and the very high AUC ROC values reported should be interpreted with this in mind rather than as evidence that real time reviewer assignment is a solved problem. Second, the queue level turnaround time simulation conducted during this work, intended to directly measure end to end latency reduction under the joint system compared to its individual components, did not fully isolate the mechanism by which correct reviewer assignment is expected to reduce latency in practice, namely the avoidance of misassignment and subsequent reviewer reassignment cycles. The per pull request review time estimates reported for the integrated system and for reviewer recommendation in isolation were therefore identical in our simulation, since both select the same recommended reviewer; the benefit of joint optimization in this work is demonstrated primarily through the priority ordering and ablation results in Section IV rather than through this simulation. A queue simulation that explicitly models reassignment probability following a suboptimal initial assignment is identified as important future work. Third, the evaluation is conducted on a single repository, Apache Flink, and although it is large and long lived, the generalizability of the specific feature weights and decay parameters to repositories with different contributor structures or review cultures has not been tested.

VI. Conclusion and Future Work

This paper presented a unified machine learning pipeline that addresses pull request prioritization and reviewer recommendation as a single, queue aware optimization problem rather than as two independent tasks. By combining a learned reviewer suitability model, a latency prediction model, and a set of expertise and collaboration signals drawn from prior work into a single joint priority score, and by propagating reviewer workload updates through the assignment process as the queue is processed, the system achieves substantially higher reviewer recommendation accuracy than a historical activity baseline and produces a priority ordering that correlates significantly with actual review latency.

Future work should focus on three directions. The first is a more mechanistically grounded simulation of end to end queue turnaround time, explicitly modeling the probability and cost of reviewer reassignment following a suboptimal match, in order to more directly quantify the latency benefit of joint optimization. The second is evaluation across multiple repositories with differing scale, governance structure, and review culture, to assess how well the learned feature weights generalize beyond Apache Flink. The third is an online or semi online evaluation, in which the system's recommendations are deployed alongside, rather than only compared retrospectively against, the decisions of human maintainers, to measure adoption and real world latency impact directly.

References

[1] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in *Proc. 36th Int. Conf. Software Engineering (ICSE)*, 2014, pp. 345–355.

[2] Y. Yu, H. Wang, G. Yin, and T. Wang, "Reviewer recommendation for pull-requests in GitHub: What can we learn from code review and bug assignment?" *Information and Software Technology*, vol. 74, pp. 204–218, 2016.

[3] P. Thongtanunam, C. Tantithamthavorn, R. G. Kula, N. Yoshida, H. Iida, and K. Matsumoto, "Who should review my code? A file location-based code-reviewer recommendation approach for modern code review," in *Proc. 22nd IEEE Int. Conf. Software Analysis, Evolution, and Reengineering (SANER)*, 2015, pp. 141–150.

[4] M. B. Zanjani, H. Kagdi, and C. Bird, "Automatically recommending peer reviewers in modern code review," *IEEE Transactions on Software Engineering*, vol. 42, no. 6, pp. 530–543, 2016.

[5] M.-S. Moosareza and A. Heydarnoori, "Automated code reviewer recommendation for pull requests," *e-Informatica Software Engineering Journal*, vol. 18, no. 1, p. 240108, 2024.

[6] Y. Yu, H. Wang, G. Yin, T. Wang, and C. X. Ling, "Who should review this pull-request: Reviewer recommendation to expedite crowd collaboration," in *Proc. 21st Asia-Pacific Software Engineering Conf. (APSEC)*, 2014, pp. 335–342.

[7] X. Xia, D. Lo, X. Wang, and B. Zhou, "TIRR: A code reviewer recommendation algorithm with topic model and reviewer influence," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2022.

[8] A. Bosu, M. Greiler, and C. Bird, "Characteristics of useful code reviews: An empirical study at Microsoft," in *Proc. 12th Working Conf. Mining Software Repositories (MSR)*, 2015, pp. 146–156.

[9] D. M. Soares, M. de Lima Júnior, L. Murta, and A. Plastino, "Rejection factors of pull requests filed by core team developers in software projects with high acceptance rates," in *Proc. 14th Int. Conf. Machine Learning and Applications (ICMLA)*, 2015, pp. 960–965.

[10] U. H. Khan, H. Ali, A. Zia, H. Khan, U. Tanveer, and S. Bibi, "Prioritizing pull requests through dual prediction of acceptance and integrator response," *Spectrum of Engineering Sciences*, vol. 3, no. 7, pp. 1701–1715, 2025.

[11] H. Raskin, X. Devroey, and B. Vanderose, "PRECOG: Pull request prioritization and visualization," in *Proc. 2026 IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2026.

[12] P. Weissgerber, D. Neu, and S. Diehl, "Small patches get in!" in *Proc. 5th Working Conf. Mining Software Repositories (MSR)*, 2008, pp. 67–76.

[13] R. Mirsaeedi and P. C. Rigby, "Nudge: Accelerating overdue pull requests toward completion," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1994–2005.